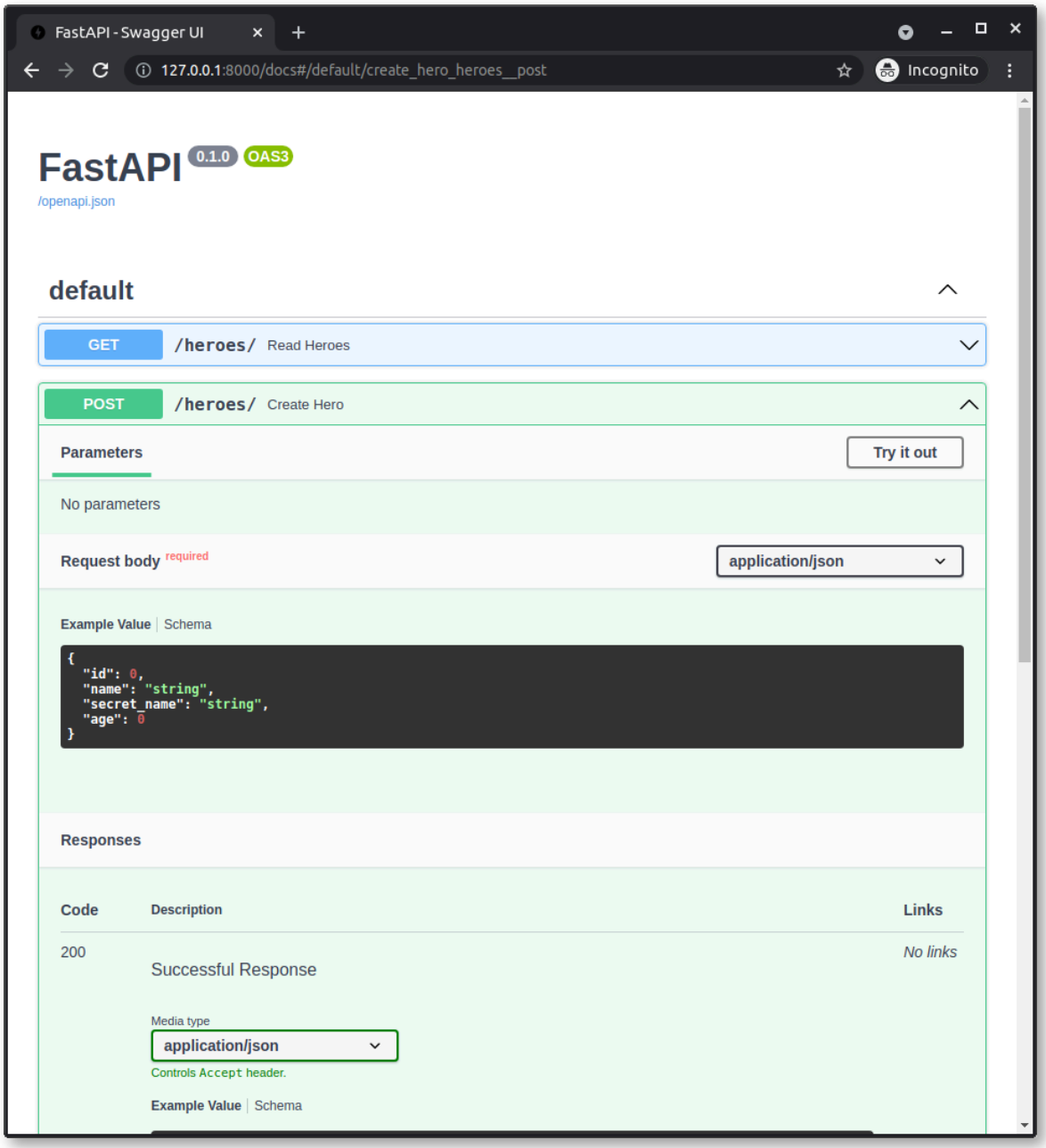
FastAPI Response Model with SQLModel

Now I'll show you how to use FastAPI's `response_model` with **SQLModel**.

Interactive API Docs

Up to now, with the code we have used, the API docs know the data the clients have to send:



This interactive docs UI is powered by Swagger UI, and what Swagger UI does is to read a big JSON content that defines the API with all the data schemas (data shapes) using the standard OpenAPI, and showing it in that nice UI.

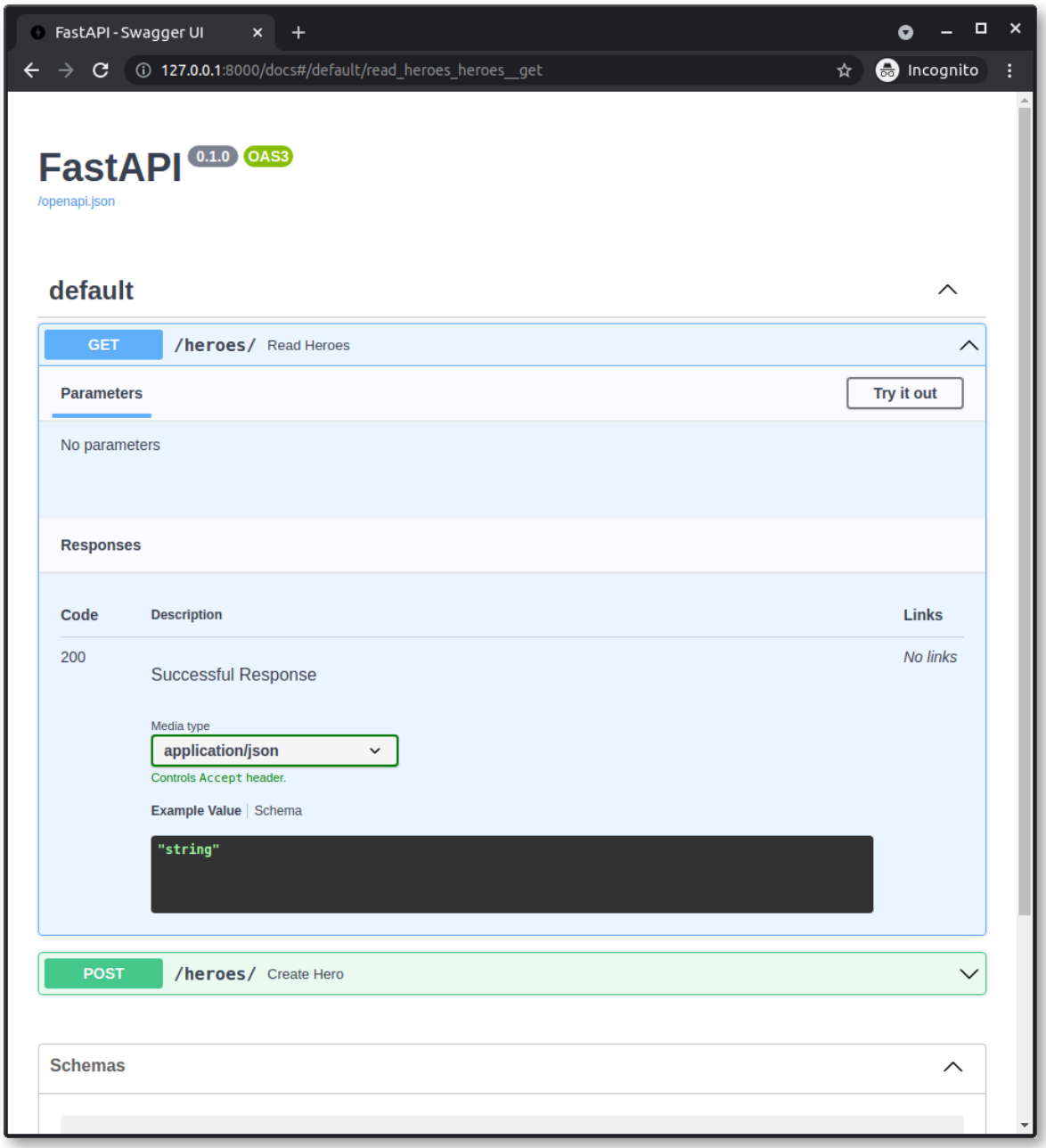
FastAPI automatically **generates that OpenAPI** for Swagger UI to read it.

And it generates it **based on the code you write**, using the Pydantic models (in this case **SQLModel** models) and type annotations to know the schemas of the data that the API handles.

Response Data

But up to now, the API docs UI doesn't know the schema of the *responses* our app sends back.

You can see that there's a possible "Successful Response" with a code `200`, but we have no idea how the response data would look like.



Right now, we only tell FastAPI the data we want to receive, but we don't tell it yet the data we want to send back.

Let's do that now. 🤓

Use `response_model`

We can use `response_model` to tell FastAPI the schema of the data we want to send back.

For example, we can pass the same `Hero` **SQLModel** class (because it is also a Pydantic model):

Python 3.10+

```
# Code above omitted 👆

@app.post("/heroes/", response_model=Hero)
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=Hero)
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/", response_model=list[Hero])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

List of Heroes in `response_model`

We can also use other type annotations, the same way we can use with Pydantic fields. For example, we can pass a list of `Hero`s.

To do so, we declare the `response_model` with `list[Hero]`:

Python 3.10+

```
# Code above omitted 👆

@app.get("/heroes/", response_model=list[Hero])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

👀 Full file preview

Python 3.10+

```
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=Hero)
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/", response_model=list[Hero])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

FastAPI and Response Model

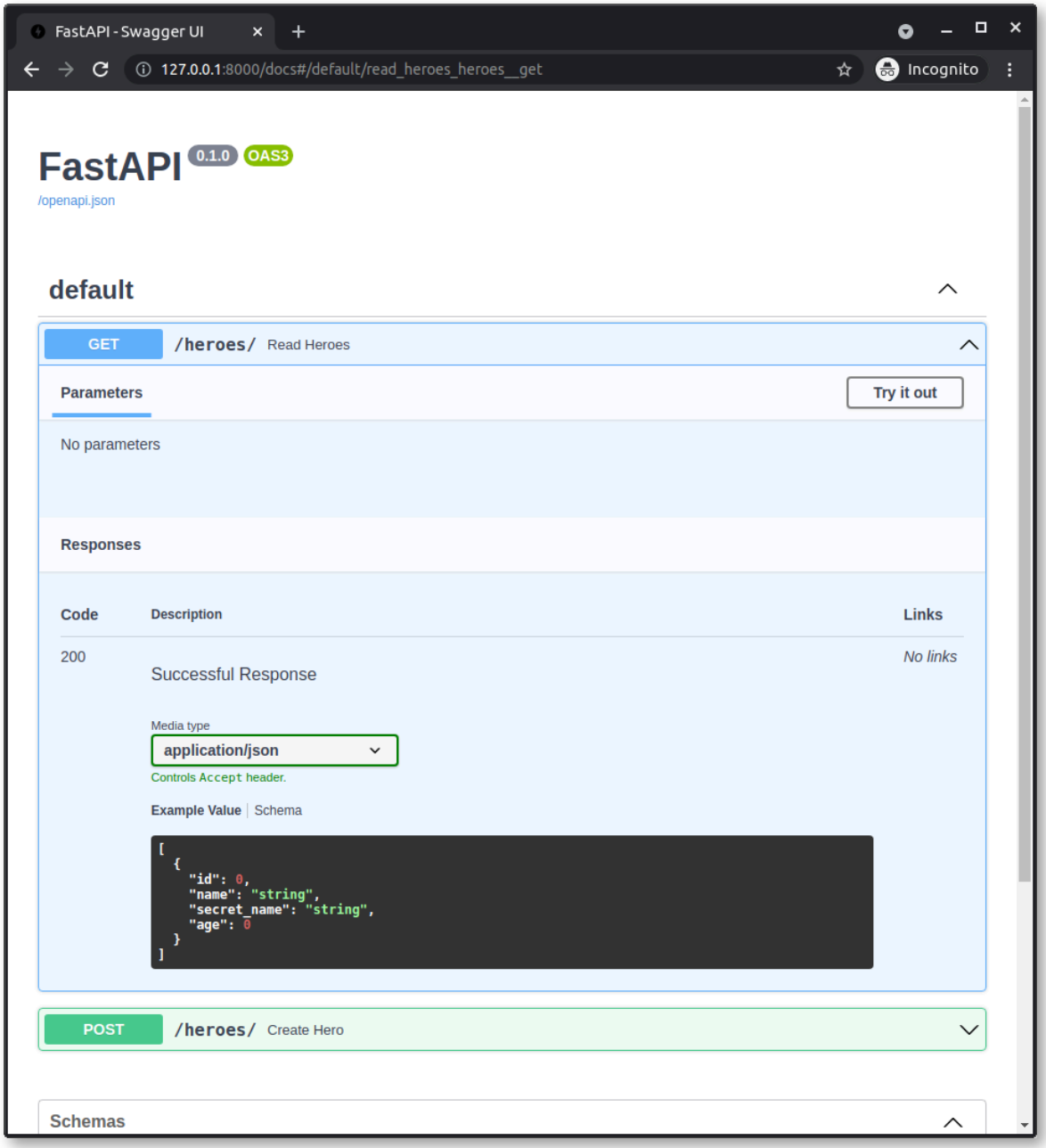
FastAPI will do data validation and filtering of the response with this `response_model`.

So this works like a contract between our application and the client.

You can read more about it in the FastAPI docs about `response_model`.

New API Docs UI

Now we can go back to the docs UI and see that they now show the schema of the response we will receive.



The clients will know what data they should expect.

Automatic Clients

The most visible advantage of using the `response_model` is that it shows up in the API docs UI.

But there are other advantages, like that FastAPI will do automatic data validation and filtering of the response data using this model.

Additionally, because the schemas are defined in using a standard, there are many tools that can take advantage of this.

For example, client generators, that can automatically create the code necessary to talk to your API in many languages.

> **Info**
>
> If you are curious about the standards, FastAPI generates OpenAPI, that internally uses JSON Schema.
>
> You can read about all that in the FastAPI docs - First Steps.

Recap

Use the `response_model` to tell FastAPI the schema of the data you want to send back and have awesome data APIs. 😎